

Where the Tokens Go: Decomposing and Closing the Decode Gap for a 27B Hybrid LLM on Apple Silicon

Anonymous submission

Draft, September 2026

Abstract

Open-weight language models with 20–30B parameters now fit on personal machines, including Apple silicon systems with large unified memory. For interactive use, their utility is determined by the latency of a single decode stream. This latency is commonly modeled as a memory-bandwidth problem: each token requires reading the active model weights, so decode time should approach model size divided by achievable bandwidth. In practice, it does not. For Qwen3.8-27B in Q4_K on an M2 Ultra, streaming the weights at the best rates observed for each weight format would take about 24 ms, yet llama.cpp takes 42.3 ms per decode forward. Existing performance models and standard profilers do not explain this gap.

We develop a measurement methodology that decomposes decode latency across GPU stages and CPU work, with matched timing definitions across engines, and validates changes against reference logits at chosen decode steps. It shows that a 27B step is about a thousand small kernels whose dispatch overhead alone costs milliseconds, that the matrix-vector kernels stream below the best rate because of activation traffic rather than occupancy or dequantization, and that CPU work between tokens is a first-order term. Guided by these measurements, we add the model to DS4, an open-source model-specific C/Metal engine, and optimize it behind runtime switches, reducing decode time from 35.0 to 30.8 ms per step; its forward takes 30.6 ms versus 42.3 ms for llama.cpp, with logits that agree at eight tested decode steps and context lengths. Two results reach beyond this engine. With the same checkpoint in Q4_0 and Q4_K, the cheaper block format is worth 8–16% under simple kernels and 2–3% once the large matvecs are fused. A 2.2 ms per-token sampler cost, an `isfinite` left as a library call under `-ffast-math`, is invisible to every GPU-side tool. Single-stream latency on unified-memory GPUs cannot be understood from bandwidth alone: it depends on how quantization format, GPU kernels, dispatch and CPU execution interact. We release the implementation, measurement tools, logs and scripts.

1 Introduction

Open-weight language models in the 20–30B parameter range now fit on personal machines with 64–192 GB of unified memory, making local inference practical for chat, coding agents, and privacy-sensitive workloads. For these interactive uses, the relevant cost is often not aggregate throughput but the latency of a single decode stream, paid once for every generated token. Apple silicon is an important platform for this workload because its CPU and GPU share a unified memory pool large enough to hold models of this size. Its GPU is programmed through Metal, however, so many CUDA-specific optimization and profiling assumptions do not transfer directly. This paper asks a simple question: how fast can such a machine decode a 27B model, and what prevents it from going faster?

Three widely used or representative Metal-based engines expose different points in the design space. llama.cpp [8] constructs a generic computation graph from the GGUF model and executes it

through ggml’s Metal backend, typically mapping graph nodes to kernels specialized by operation and quantization type. MLX [10] is Apple’s array framework; mlx-lm expresses the model in Python over lazily evaluated primitives, some of which are fused, and uses its own affine quantization. We also use and extend DS4 [21] (DwarfStar), an open-source C/Metal inference engine developed by Salvatore Sanfilippo and contributors (<https://github.com/antirez/ds4>). Unlike the generic graph-based engines, DS4 implements each model family as a hand-written execution schedule, making model-specific fusion and scheduling changes local and easy to isolate experimentally.

A common mental model for single-stream decode is that it is dominated by memory bandwidth. Each token requires reading the active model weights, so step time should approach model bytes divided by achievable bandwidth. For Qwen3.8-27B in Q4_K, one decode step reads 15.06 GB of weights. That implies 18.8 ms at the M2 Ultra’s nominal 800 GB/s, or about 24.3 ms at the best streaming bandwidth we measure for the relevant weight formats on this GPU (§2.3). Yet llama.cpp takes 42.3 ms per decode step (§7). The simple bandwidth model therefore explains only a little over half of the observed latency.

The missing time is not straightforward to recover with standard tools. Instrumenting stage boundaries with synchronization adds about 0.16 ms per boundary on this machine, more than the execution time of many of the kernels being measured. System traces report the GPU as 96–98% busy for both llama.cpp and our engine, but high occupancy in time does not imply that the running kernels are moving data at the streaming limit. A kernel can keep the GPU busy while using memory bandwidth inefficiently. GPU-side tools also miss CPU work between tokens, even though that work lies directly on the single-stream critical path.

We therefore begin with measurement. Our instrumentation (§3) records GPU start and end timestamps for named stages directly from the command buffer, decomposes CPU time between tokens into forward, sampling, and emission, applies matched timing definitions across engines, and compares logits against llama.cpp at selected decode steps. This lets us localize latency without perturbing the execution path enough to dominate the measurements, while checking that each optimization preserves decode behavior.

Applying the instrumentation to the pre-optimization engine (§4) reveals four effects that organize the rest of the paper.

First, **dispatch is a first-order cost**. A 27B decode step is not one large streaming operation but roughly a thousand small GPU kernels. At this granularity, launch, ramp, and drain costs accumulate to several milliseconds per token.

Second, **the dominant matrix-vector kernels do not reach the best streaming bandwidth available on the device**. Controlled ablations show that the important limiter is activation traffic rather than occupancy or dequantization cost. This distinction matters because an optimization aimed at arithmetic or thread count does little if the kernel is rereading activations unnecessarily.

Third, **quantization format and kernel implementation cannot be evaluated independently**. Using the same checkpoint in Q4_0 and Q4_K, we find that the cheaper Q4_0 format is 8–16% faster with simple kernels but only 2–3% faster once the large matrix-vector operations are fused (§7.4). The value of a representation therefore depends on the kernels that consume it, not only on its nominal bytes per weight.

Fourth, **the CPU remains on the critical path**. In the initial implementation, sampling alone costs 2.2 ms per token, because `isfinite` is left as a library call under `-ffast-math`; llama.cpp’s own full-vocabulary top- p sampler costs 1.2 ms per token under the same settings (§5.3). This cost is invisible to GPU profiling, yet it is large enough to materially affect end-to-end latency. More generally, a GPU-side accounting of decode is incomplete unless the inter-token CPU path is measured separately.

Guided by these observations, we optimize DS4 along three dimensions (§5): weight representation and matrix-vector data movement, dispatch count and kernel fusion, and CPU work between tokens. Each optimization is guarded by a runtime switch, so the final implementation can reproduce any subset of the changes and measure their effects in isolation. To separate representation effects from kernel effects, we also build Q4.0 and Q4.K versions from the same checkpoint using the same importance matrix and output head, and evaluate both formats at multiple fusion levels (§7.4).

With all optimization switches disabled, a decode step in DS4 takes 35.0 ms. Enabling them reduces the step to 30.8 ms in the generation-loop definition, or 30.6 ms per forward call, against 42.3 ms for llama.cpp under the same forward definition. The 12% improvement from 35.0 to 30.8 ms is a controlled within-engine result using the same model and weight file. The larger gap to llama.cpp is a comparison between complete engines and is not attributed to any single mechanism. Across eight decode-step and context-length validation points, the optimized engine’s logits agree with llama.cpp’s.

The paper makes the following contributions:

1. **A measurement methodology for single-stream Metal decode** (§3, §6) that combines per-stage GPU timing, CPU inter-token decomposition, matched engine timing definitions, and decode-step correctness checks.
2. **An empirical diagnosis and optimization of 27B local inference on Apple silicon** (§4, §5), identifying dispatch overhead, sub-streaming matrix-vector behavior, representation/kernel interactions, and CPU work as distinct components of the latency gap. Guided by these measurements, we reduce DS4 decode latency from 35.0 to 30.8 ms per decode forward and the sampler’s cost from 2.2 to 0.41 ms per token.
3. **A controlled study of quantization format and kernel interaction** (§7.4), showing that the performance advantage of Q4.0 over Q4.K shrinks substantially as matrix-vector kernels improve, and measuring the corresponding quality tradeoff; and a context-length sweep against llama.cpp and MLX (§7), where MLX on its affine 4-bit conversion lands within about 1 ms of our step.
4. **A reproducible experimental artifact**, including switchable optimizations, negative results and their mechanisms (§8), profiling support for additional model classes, logs, and analysis scripts (Appendix B).

Our DS4 branches, scripts, and logs are available at <https://github.com/parasol-aser/ds4>.

2 Background

2.1 Platform and setup

The M2 Ultra CPU and GPU share an LPDDR5 memory pool. Metal compiles kernels at runtime and executes them from command buffers; a compute encoder holds many dispatches, which execute in order with memory ordering. There is no host-to-device copy, but the CPU and the GPU share bandwidth, and CPU work between two tokens is on the critical path.

All measurements are on a Mac Studio with an M2 Ultra (24 CPU cores, 60 GPU cores, 192 GB LPDDR5, 800 GB/s nominal), macOS 26.1, Apple clang 17.0.0 (clang-1700.4.4.1). Baselines are llama.cpp at commit `6d9c82ea2bb34e277c0664b8dd3434bfb4dcfb27` (2026-09-09) built with its Metal backend [8], and mlx-lm 0.31.3 on mlx 0.32.2 [17]. The GGUF engines read the same files from the Unsloth release (revision `4ca7207` [22]); MLX uses the community 4-bit conversion (revision `3e6447f`) of the same checkpoint (revision `1d4bf0f` [20]). Appendix B lists file hashes.

Component	Weights	Q4_K bytes
DeltaNet layer ($\times 48$): qkv, z, α, β , out, FFN	383.3M	215.6 MB
Attention layer ($\times 16$): $q gate, k, v$, out, FFN	372.3M	209.4 MB
Decoder layers, from tensor sizes		13.71 GB
Output head, Q8_0 (248320×5120)	1.27B	1.35 GB
Per decode step		15.06 GB

Table 1: Bytes read per decode step for Qwen3.8-27B in pure Q4_K (0.5625 B/weight) with a Q8_0 output head (1.0625 B/weight), from the GGUF tensor sizes. The output norm adds 20 KB; the auxiliary MTP block (239 MB) and the token-embedding table (715 MB, one row read) are excluded.

2.2 Model

Qwen3.8-27B (GGUF architecture `qwen35`) has 64 decoder layers plus one auxiliary multi-token-prediction block that ordinary decode does not execute; the GGUF therefore has 65 blocks, and the byte accounting below covers the 64 decoder layers, the output norm and head, and one row of the token embedding. In three layers out of four the token mixer is Gated DeltaNet [23]: one fused projection produces q and k (16 heads of 128) and v (48 heads of 128), which pass through a causal depthwise convolution of width 4 followed by SiLU; q and k are L2-normalized per head and q is scaled by $1/\sqrt{128}$; value head h uses query/key head $h \bmod 16$; each value head keeps a 128×128 recurrent state updated by the delta rule with a scalar decay $\exp(A \cdot \text{softplus}(\alpha + b))$, where $A = -\exp(A_{\log})$ is a per-head parameter, b a per-head bias and α comes from a 48-wide projection, and a write strength $\sigma(\beta)$ from a second 48-wide projection; the output is RMS-normalized per head and gated by SiLU(z) from a 6144-wide projection. Every fourth layer is gated grouped-query attention: 24 query heads of dimension 256 with 4 KV heads, per-head RMSNorm on q and k , NEOX RoPE on the first 64 dimensions, and the output multiplied by a sigmoid gate projected alongside q . All layers have a SwiGLU FFN of hidden size 17408; the width is 5120 and the vocabulary has 248,320 entries. Table 1 gives the bytes a decode step reads.

2.3 Streaming bound

At 15.06 GB per step the nominal 800 GB/s bound is 18.8 ms (53 t/s). A more useful reference applies the fastest rate we measure for each format to its bytes: the Q8_0 output-head matvec moves 1.35 GB in 2.01 ms (670 GB/s), and the fastest Q4_K kernel, the fused gate/up projection, moves 100.3 MB in 162.9 μ s (616 GB/s). Then

$$T_{\text{weights}} \approx \frac{13.71 \text{ GB}}{616 \text{ GB/s}} + \frac{1.35 \text{ GB}}{670 \text{ GB/s}} \approx 24.3 \text{ ms},$$

which we call the estimated weight-streaming bound. It assumes every matrix shape streams at the best measured rate for its format and excludes work outside the matvecs. Because the rates come from complete kernels, they already include dequantization and kernel overhead. The estimate is an empirical reference, rather than a hardware lower bound or a partition of measured execution time. Throughout, “utilization” of this reference means 24.3 ms divided by the measured step time; it is not a measurement of DRAM bandwidth utilization.

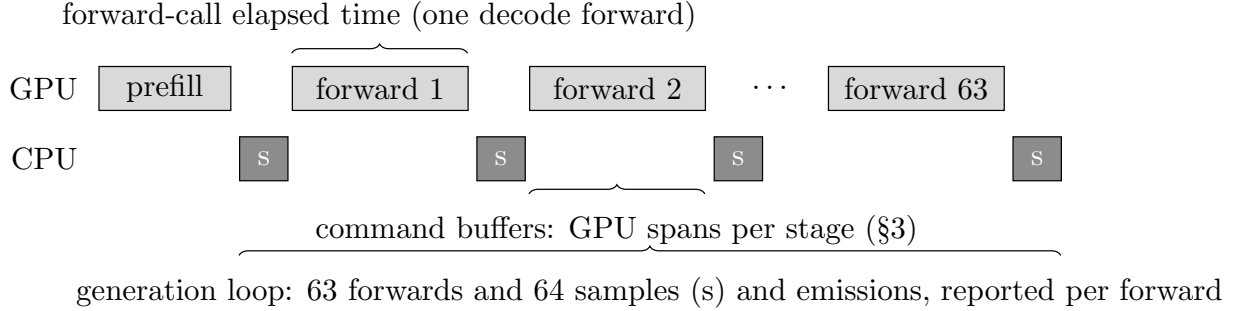


Figure 1: What each reported quantity covers. Token 1 is sampled from the prefill logits, so a 64-token run performs 63 decode forwards. We report milliseconds per decode forward under two definitions: *forward-only*, the host-elapsed time of one forward call (encoding, submission and synchronization included; distinct from the per-stage GPU spans of §3), and *generation loop*, the whole loop (forward, sampling, emission) divided by the 63 forwards. For DS4 both come from the CLI under DS4_TOKEN_TIMING; for llama.cpp the forward is its *eval time* per run and the loop is approximated by adding its *sampling time*, which omits its emission overhead; mlx-lm reports only n output tokens over the time of $n - 1$ forwards, converted accordingly. Rates in t/s are 1000/ms.

3 Measurement method

Timing definitions. Engines count differently: a 64-token run performs 63 decode forwards, the first token being sampled from the prefill logits, and llama.cpp’s *eval time* excludes sampling while a generation-loop clock includes it. We therefore report milliseconds per decode forward under two explicit definitions (Figure 1), averaging over all 63 forwards of a run in both engines. Headline numbers are means over five runs (three for llama.cpp’s default-sampler column) on an otherwise idle GPU with the model file in the page cache; standard deviations are in the result set.

Per-stage GPU spans. DS4 can close and reopen its command buffer at named stage boundaries inside a layer under an environment switch. We time each stage with the command buffer’s GPUStartTime and GPUEndTime. Timing the same boundaries with the CPU clock includes a synchronization of about 0.16 ms per boundary, which is larger than the recurrence, the attention or any norm. Isolating stages costs wall-clock time (the instrumented run decodes at 8.7 t/s) but the GPU spans themselves are little affected: their sum, 31.1 ms, is within 2% of the uninstrumented forward (30.6 ms). We use the profile for ranking and for deltas.

System traces. Metal System Trace (xctrace) records GPU busy intervals and command-buffer submissions process-wide; from it we take GPU busy fraction and CPU encode time per token, for DS4 and llama.cpp on the same prompt.

Reference logits. A small program on the llama.cpp API prefills a prompt, greedy-decodes, and writes the full logit vector at step k , where step 0 is the prefill output and step k is the output after feeding back k generated tokens; DS4 writes the same under a switch. We report mean and maximum absolute difference over the 248,320 entries, the KL divergence of the softmax distributions, and argmax agreement, at steps 0, 4, 16, 40 and 63 on a 20-token prompt, at steps 0, 4 and 63 on a 1,439-token prompt, and at step 4 on an 8,590-token prompt. With greedy decoding on both sides

Layer	Stage	Optimizations off		On		On, 8.6k ctx ms/step
		μ s/layer	ms/step	μ s/layer	ms/step	
both	FFN gate/up	195.6	12.52	162.9	10.43	10.44
both	FFN down	112.0	7.16	95.0	6.08	6.07
DeltaNet	projections qkv, z, α, β	115.2	5.53	94.1	4.52	4.48
attention	projections $q gate, k, v$	94.9	1.52	82.1	1.31	1.30
DeltaNet	output norm + out projection	45.3	2.17	48.3	2.32	2.31
attention	gate + out projection	42.8	0.68	44.9	0.72	0.72
DeltaNet	conv + recurrence	23.8	1.14	23.8	1.14	1.14
attention	split-KV attention (+pad, reduce)	106.3	1.70	60.3	0.96	3.36
attention	q/k prologue, V store	7.9	0.13	8.0	0.13	0.14
both	residual adds and norms	29.9	1.91	23.6	1.51	1.48
	output head (Q8.0)		2.01		2.01	2.01
sum of stage spans			36.47		31.13	33.47
uninstrumented step (loop definition)			35.0		30.8	32.6

Table 2: GPU span per stage, Qwen3.8-27B pure Q4_K, M2 Ultra, averaged over decode positions 21–30 of the 20-token prompt (the “8.6k” column over positions 8,591–8,600 of the 8,590-token prompt). “Off” disables the six switches of §5; the single-launch prologue is present in both columns. Each stage runs in its own command buffer; the “both” rows give the per-layer value averaged over all 64 layers.

the histories are identical up to k only while the argmax agreed at every earlier step, so we also compare the 64-token greedy outputs directly and report a step only when the histories still agree.

4 Anatomy of a decode step

Table 2 is the per-stage profile with every optimization switched off and on, and Figure 2 groups it. Three observations organize the rest of the paper.

The large matvecs are near, not at, the bound. The FFN gate/up pair moves 100.3 MB per layer. As two ggml-style matvecs plus a SwiGLU kernel its effective rate is 513 GB/s; fused and vectorized (§5), 616 GB/s. The 5120-output projections (FFN down, attention and DeltaNet output) run at 390–450 GB/s with the ggml-style kernel. The Q8.0 output head reaches 670 GB/s on the same memory system. Effective rate is bytes over kernel time, not a measurement of DRAM traffic; the difference between formats is consistent with the ALU cost of unpacking Q4_K’s 6-bit sub-block scales and minimums. Section 7.4 compares the formats under different kernel configurations to examine their interaction.

Dispatch count matters at this scale. The initial model implementation issued about 1,070 dispatches per step (16 per DeltaNet layer, 19 per attention layer, plus embedding and head). At a few microseconds of ramp and drain each, that is 3–5 ms, as much as the recurrence, the attention and all norms together. The final graph issues about 620.

GPU busy is not GPU useful. On the mixed-recipe UD-Q4_K_M file, the system trace shows DS4 at 98.3% GPU busy against 95.6% for llama.cpp, with about 1 ms of CPU encode time per token against 4.9 ms. Both engines keep the GPU occupied; the gap to the bound is mostly inside kernels, not between them, which is why the work below is mostly about kernel shape and fusion.

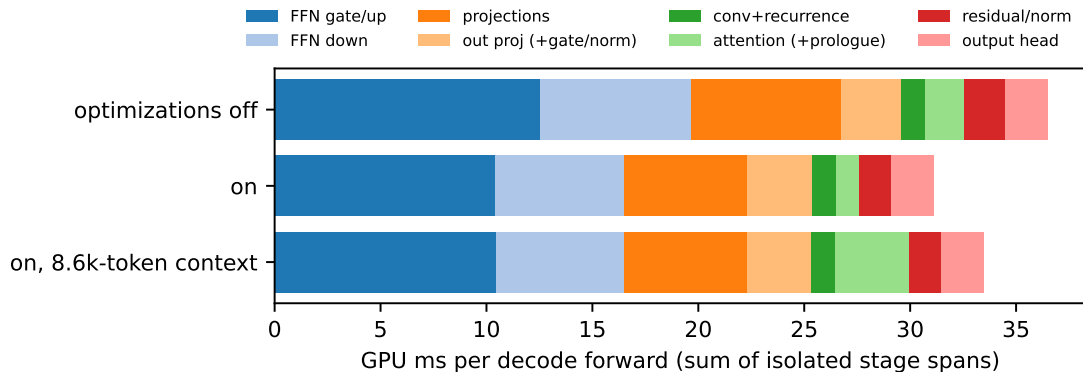


Figure 2: Table 2 grouped by stage type.

5 Optimizations by lever

Six optimizations have runtime switches (Appendix B lists them); Figure 3 and Table 8 give their leave-one-out effects around the optimized build, in milliseconds per decode forward. Leave-one-out effects around the final build need not equal standalone gains from the original build, and several switches remove the same dispatch overhead, so they do not sum to the all-off difference.

5.1 Bytes: how fast a byte moves

The format fixes the byte count; the kernel decides the effective rate. Two changes to the Q4_K inner loop were worth 9.6% on the fused gate/up kernel (178.6 to 162.9 μ s per layer): staging the 32 activation values a lane needs as eight `float4` loads instead of 32 scalar loads (the activation pointer’s alignment is not provable to the compiler, so it does not vectorize the loads itself), and loading each lane’s two 8-byte nibble runs as `uint2` instead of four `uint16`. Amortizing one activation staging over four rows per simdgroup instead of two helped the 10240- and 12288-output projections in the stage profile and hurt the 5120-output ones, so the row count is chosen per shape; end to end the difference is within noise. A K-split variant, which spreads one row’s reduction over the simdgroups of a threadgroup and combines through threadgroup memory, helps only the long-row FFN down projection (17408 inputs: 112 to 95 μ s) and is used only there.

5.2 Dispatches: fewer and better shaped

Multi-weight matvec. A DeltaNet layer’s four side projections (qkv , z , α , β) share one activation and one input width. One kernel with up to four weight segments, dispatched as a single grid whose threadgroup ranges map to segments, writes the outputs back to back into the existing batch buffer; the convolution and recurrence read z , α and β through views. The same kernel issues $q|gate$, k and v of an attention layer. Four launches become one, and the 48-output α and β heads no longer each pay a dispatch.

Fused gate/up SwiGLU. One kernel streams the gate and up weights against one staged activation and writes $\text{SiLU}(g) \cdot u$; the elementwise pass disappears and the activation is read once per four rows instead of once per two.

Residual add fused into the next norm. Each residual add is fused with the RMSNorm that consumes its result, so a layer’s FFN epilogue produces the next layer’s attention-norm input; two adds and two norms per layer become two kernels.

Single attention prologue. Per-head RMSNorm plus RoPE for the query heads, the same for the key heads into the f16 K cache, and the V store are one grid with three kinds of threadgroup instead of three launches (this one has no switch).

Context-scaled split-KV. The split-KV decode attention kernel adapted from ggml partitions the keys over 32 workgroups and reduces their partials in a second kernel. At a 20-token context 31 workgroups are idle and the reduce dominates. Scaling the workgroup count with the context length, and letting a single workgroup write its normalized output directly, cuts the attention stage from 106 to 60 μ s per layer; the shared reduce kernel had silently assumed 32 workgroups and was made correct for fewer.

5.3 The CPU path between tokens

Sampling settings first, since they decide what the sampler has to do. The model’s generation configuration specifies temperature 1.0, top- p 0.95 and top- k 20; llama.cpp reads these from the file’s metadata and its default chain applied top- k 20 and min- p 0.05 in our runs. DS4’s Qwen defaults are temperature 1.0 and top- p 0.95 with no top- k and no min- p , so its sampler scans the full 248,320-entry distribution every token; this is the configuration studied here, and for comparison we also ran llama.cpp with the same settings (`--top-k 0 --min-p 0`).

Under those settings the CLI spent 2.2 ms per token in DS4’s sampler against 30.5 ms in the forward. The cost was not the softmax or the sort but `isfinite()` on each logit, twice per token: the engine is compiled with `-ffast-math`, under which Apple clang 17 leaves `isfinite` as a call to `__isfinitef` and the scan runs scalar at about 4 ns per element. A first replacement, an exponent-bit test on the float value, was folded to “always finite” by the optimizer under the same flags, admitted NaN and $-\infty$ logits, and failed 1,459 unit tests; the working version tests the bits as loaded from memory, which the optimizer does not classify. The finite test alone did not recover the time; the gain came from rewriting the max scan and the top- p pass branch-free and block-wise so they vectorize, with a degree-5 fit of 2^x (relative error under 2×10^{-6}) for the tail mass that cannot enter the 512-entry candidate heap, and a lowering-cutoff fallback instead of a full-vocabulary sort. The sampler now costs 0.41 ms per token. The flags cut both ways: the same sampler built with `-fno-finite-math-only`, which inlines `isfinite`, costs 0.74 ms, because the vectorized scans lose the fast-math assumptions. For scale, llama.cpp’s sampler costs 0.13 ms per token with its default top- k 20 chain and 1.21 ms with the full-vocabulary top- p settings. Two lessons are general: measure the inter-token gap on the CPU, and treat fast-math flags as part of the numerical contract of code that handles non-finite values. Whether other compilers or SDK versions behave the same way we have not checked.

6 Correctness

Table 3 compares DS4 with llama.cpp on the same Q4_K file at eight (prompt, step) points where the greedy histories still agree. The argmax agrees at every point and the distributions are close; the 64-token greedy outputs are identical on the Q4_0, UD-Q4_K_M and Q8_0 files and diverge at token 46 on Q4_K, where a near-tie resolves differently (so the step-63 point on the 20-token prompt

Prompt	step	mean $ \Delta $	max $ \Delta $	KL	argmax
20-token prompt	0	0.0377	0.299	2.3e-05	agrees
20-token prompt	4	0.0042	0.045	1.7e-07	agrees
20-token prompt	16	0.0042	0.045	1.2e-08	agrees
20-token prompt	40	0.0061	0.055	8.3e-09	agrees
1,439-token prompt	0	0.0900	0.522	4.2e-05	agrees
1,439-token prompt	4	0.0243	0.213	2.0e-06	agrees
1,439-token prompt	63	0.0138	0.199	3.5e-05	agrees
8,590-token prompt	4	0.0153	0.205	2.9e-07	agrees

Table 3: Logit difference from llama.cpp, pure Q4_K, over 248,320 entries; KL is $D(\text{DS4} \parallel \text{llama.cpp})$ of the softmax distributions.

File	bytes/weight	PPL	paired vs pure Q4_K
pure Q4_K, imatrix	0.5625	5.80 ± 0.07	
UD-Q4_K_M	0.58	5.85 ± 0.07	$+0.8\% \pm 0.4$
Q8_0	1.0625	5.86 ± 0.07	$+1.0\% \pm 0.5$
pure Q4_0, imatrix	0.5625	6.03 ± 0.07	$+3.9\% \pm 0.9$
MLX 4-bit affine, group 64	0.5625	5.98	

Table 4: Perplexity on the first 40 chunks of 2048 tokens of wikitext-2 test, scoring positions 1025–2047 of each chunk (**llama-perplexity**; a script with the same chunking for the MLX model). The $\pm$ after each PPL is llama-perplexity’s token-level standard error; the last column is the relative perplexity change $\exp(\bar{d}) - 1$ from the mean paired difference $\bar{d}$ of per-chunk loss, with the delta-method standard error $\exp(\bar{d}) \text{SE}(\bar{d})$ over 40 chunks. Bytes per weight are for the layer tensors.

compares different histories and is omitted). The values were unchanged, to the printed precision, across every optimization added in §5; prefill logits are bit-identical between the initial and final builds because no prefill kernel changed. Three prompts and eight (prompt, step) points are spot checks, not a proof of equivalence over long generations.

6.1 Quality of the files

We measure the quantization quality of the GGUF files with llama.cpp; the logit comparisons above separately establish DS4’s numerical agreement with that implementation on the tested prompts and steps, and do not by themselves prove that quality is engine-independent along long generations. Table 4 gives wikitext-2 perplexity [16], and because all files score the same chunks we report the paired relative perplexity change with a chunk-level standard error, which is the informative statistic. Pure Q4_K, UD-Q4_K_M and Q8_0 are within 1% of each other (the differences are 1–2 chunk-level standard errors, and the ordering is not the one bit width would predict); pure Q4_0 and the MLX conversion are 3–4% worse, at 4 standard errors for Q4_0. Forty chunks of wikitext-2 are a narrow view of quality.

File	fwd	DS4 (ms/forward)		fwd	llama.cpp (ms/forward)		prefill t/s
		loop greedy	loop sampled		loop default	loop matched	
pure Q4_K	30.6	30.7	31.1	42.3	42.4	43.6	272 / 269
pure Q4_0	30.0	30.0	30.4	35.4	35.5	36.6	289 / 304
UD-Q4_K_M	37.2	37.3	37.6	44.1	44.2	45.4	262 / 260
Q8_0	44.1	44.2	44.5	49.8	49.9	51.1	305 / 303

Table 5: Milliseconds per decode forward, 20-token ChatML prompt with an empty thinking block, 64 generated tokens, mean of five runs (three for llama.cpp’s default column; standard deviations ≤ 0.2 ms). “loop” is the generation loop per forward: DS4 greedy and sampled with its settings (temperature 1, top- p 0.95, no top- k); llama.cpp’s “default” is its metadata-driven chain (top- k 20, min- p 0.05) and “matched” is DS4’s setting, both as eval plus sampling time. The sampled comparison is therefore DS4 “loop, sampled” against llama.cpp “loop, matched”. Prefill is t/s on the 1,439-token prompt. Rates: DS4 32.6 t/s and llama.cpp 23.7 t/s forward-only on pure Q4_K.

7 Evaluation

7.1 Decode step time

Table 5 gives the headline comparison under both timing definitions. On pure Q4_K, DS4 takes 30.6 ms per decode forward against 42.3 for llama.cpp, an approximately 28% shorter step; the estimated streaming bound of 24.3 ms is 79% of it. The step is about 26 ms of weight-carrying matvecs at 390–616 GB/s effective, 2 ms for the output head, and 2–3 ms of small kernels and per-dispatch ramp. On the mixed-recipe file the step is 16% shorter than llama.cpp’s (only its Q4_K tensors take the fused paths) and on Q8.0 11% (only the format-independent fusions apply). Sampling adds 0.4 ms per token in DS4 and 0.1 ms (default chain) or 1.2 ms (full-vocabulary top- p) in llama.cpp.

With the six switchable GPU optimizations disabled, DS4 takes 35.0 ms per forward under the generation-loop definition on pure Q4_K, compared with 30.8 ms when they are enabled (Table 8). This is a 4.2 ms (12%) reduction within the same engine and weight file. Both configurations retain the single attention prologue and the current CPU sampler. The comparison with llama.cpp measures the complete implementations; it does not isolate the causes of the remaining gap. The profiles of §4 suggest a role for dispatch count and CPU encoding, but those factors were not ablated across engines.

7.2 Ablation

The fused gate/up kernel is the largest single item, followed by the multi-weight projections; the vectorized single-weight matvec and the K-split overlap because both replace the ggml-style kernel on the FFN down projection. The residual+norm fusion and the context-scaled attention are small at a 20-token context.

7.3 Context length and the MLX baseline

Figure 4 sweeps the prompt from 1.4k to 33k tokens (values in Table 9, appendix; three runs each). DS4’s step grows from 31.4 to 38.1 ms and llama.cpp’s from 42.5 to 49.4. The DeltaNet stages vary little between the short- and 8.6k-context profiles (Table 2). The sixteen attention layers read a K/V cache of 64 KiB per position summed over those layers; their attention stage, including split-KV reduction, grows from 0.96 to 3.36 ms per step between 20 and 8,590 tokens. Prefill throughput is

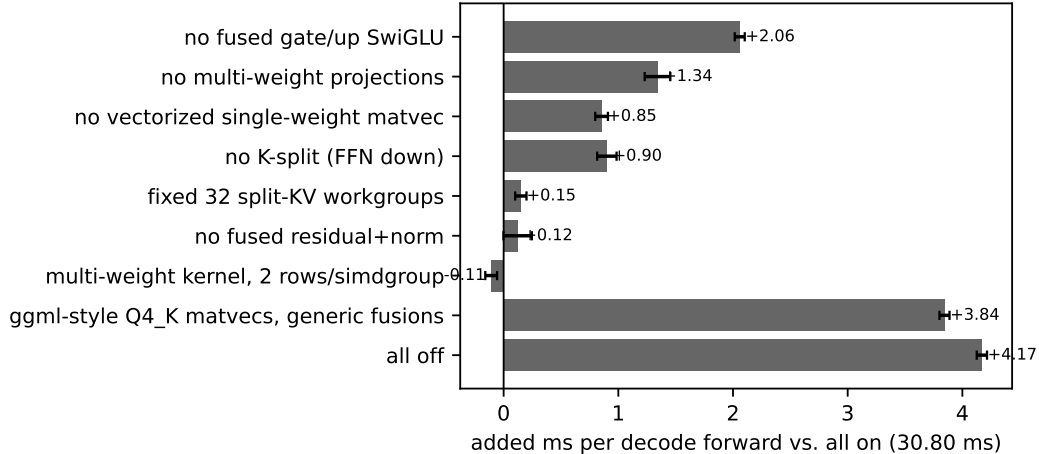


Figure 3: Leave-one-out effect of each switch on the decode step, in milliseconds per forward added relative to the all-on build; three runs each, error bars are the combined standard deviation. Table 8 in the appendix has the values.

the same in DS4 and llama.cpp within 4% at every length: prefill is a matrix–matrix workload on comparable dequantize-and-multiply kernels.

MLX is a practical baseline rather than a controlled one. Its community conversion stores the layers in 4-bit affine format (a 16-bit scale and bias per group of 64; 4.5 bits per weight, the same as Q4_K) but also the output head, at 715 MB against our Q8_0 head’s 1.35 GB, about 1 ms per step at these rates; mlx-lm reports only the generation loop; and the model code is Python over the framework’s primitives. On the same 20-token prompt its loop takes 29.8 ms per forward, comparable to DS4’s 30.7, and it tracks DS4 across context lengths (30.4 to 37.1 ms, within about 1 ms of DS4 at every length) with prefill 13–18% slower. The comparison shows that a general framework on a cheaper format reaches a similar step time; it does not isolate the format, which is what §7.4 does.

7.4 Quantization format and kernel organization

To separate the quantization format from everything else we built two files from the same Q8_0 source with the same importance matrix and the same Q8_0 output head: pure Q4_K and pure Q4_0, the same bit budget (16.0 GB on disk) with different block layouts and therefore different reconstructed values. Q4_0 stores one 16-bit scale per 32 weights and dequantizes with one multiply-add per weight; Q4_K packs 6-bit sub-block scales and minimums into a 256-weight super-block that must be unpacked. We ported the ggml-style Q4_0 matvec into DS4 and then the fused gate/up and multi-weight kernels; Table 6 gives the step time at each kernel level in both engines.

With the baseline kernels, Q4_0 shortens the step by 16% in llama.cpp and 8% in DS4. In DS4, its gate/up pair takes 171 μ s per layer against 196 for Q4_K. With the fused kernels, the two formats take 30.0 and 30.8 ms, respectively, a difference of approximately 2.5%. The fused gate/up kernel takes 166 μ s in Q4_0 and 163 in Q4_K, both about 610 GB/s effective. Thus, the simpler format provides no measured advantage on the largest projection under these fused implementations.

The remaining differences depend on matrix shape and kernel choice. Q4_0 is faster on the narrower projections (84 versus 94 μ s for the DeltaNet projections, and 75 versus 82 for attention) but slower on FFN down (103 versus 95 μ s), where only Q4_K uses a K-split kernel. These results

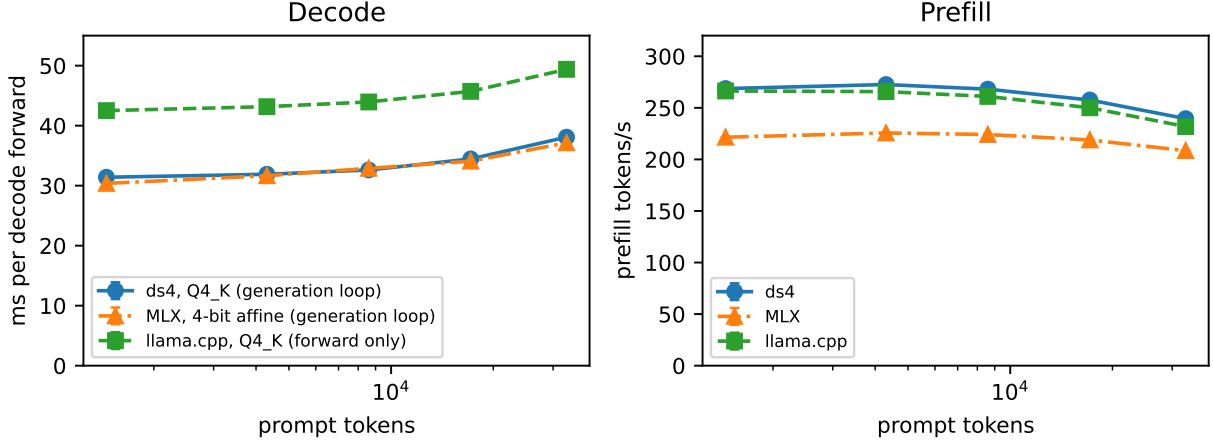


Figure 4: Decode step and prefill throughput versus prompt length, three runs each. DS4 and llama.cpp on pure Q4_K; MLX on its 4-bit affine conversion (generation loop, 4-bit head).

Engine	Kernels	Q4_K	Q4_0
llama.cpp	stock (forward-only)	42.3	35.4
DS4	per-format ggml-style matvecs, generic fusions	34.6	31.8
DS4	+ fused gate/up and multi-weight kernels	30.8	30.0
wikitext-2 PPL		5.80	6.03

Table 6: Milliseconds per decode forward for the same source checkpoint in Q4_K and Q4_0 (both pure, same imatrix, Q8.0 head), greedy, 20-token prompt; DS4 rows use the generation-loop definition (forward-only is 0.1 ms less), mean of three to five runs. “Generic fusions” are the residual+norm fusion, the single prologue and the context-scaled attention, which do not depend on the format. Q4_0 logits agree with llama.cpp to a mean $|\Delta|$ of 0.0009 at step 4, and its 64-token greedy output is identical.

support evaluating the format and its implementation together; they do not isolate dequantization instruction cost from the other kernel differences. The smaller latency advantage after fusion is accompanied by approximately 4% higher perplexity for Q4_0 on the evaluated chunks.

7.5 A second family: mixture of experts

DS4 was written for DeepSeek V4 Flash, a mixture-of-experts model, and its decode there is shaped by a different lever: only the routed experts of a token are read, so bytes per step depend on routing, and the engine gathers expert weights from a combined buffer and fuses the router with the selection. On the 86 GB mixed-recipe file (IQ2_XXS routed experts, Q8.0 attention projections, shared experts and head), DS4 takes 33.9 ms per decode forward (loop definition) against 39.6 forward-only for llama.cpp on the same file (greedy, short prompt). Table 7 is the same profiler on this model. The routed experts, the query path and the attention output projection account for two thirds of the step, and no single projection dominates the way the FFN does in the dense model; the decomposition applies per family with a different dominant term, and the specific kernels do not transfer.

Profiler stage (43 layers)	$\mu\text{s}/\text{layer}$	ms/step
routed_moe	202	8.68
q_path	165	7.10
attn_inv_rope	134	5.77
attn_output	118	5.09
router	51	2.19
kv_path	33	1.41
attn_hc_pre	27	1.14
ffn_hc_pre	27	1.15
shared_down	23	1.00
other (compressor and indexer updates, norms; amortized)		0.58
sum of stage spans		34.1

Table 7: Per-stage GPU span for DeepSeek V4 Flash decode on DS4, short prompt, decode positions 18–27, using the engine’s stage labels. A stage whose kernels are encoded in a neighbour’s command buffer is attributed to that neighbour, which is why some labels report zero and are omitted.

8 What did not work

K-split on wide matrices (measured). Splitting the reduction across the simdgroups of a threadgroup multiplies the thread count for a fixed number of output rows. It helped only the long-row FFN down projection and was neutral or slightly worse on the 10240- and 12288-output projections. This is consistent with activation re-reads per row, not occupancy, being the limiter on those shapes, since rows per simdgroup was the parameter that moved them; a change of rows per simdgroup alters several things at once, and we did not isolate the mechanism further.

Output gate fused into the attention reduce (measured). A copy of the split-KV reduce that multiplies by $\sigma(\text{gate})$ before its single store was consistently 20 μs per layer slower than the shared reduce followed by a trivial elementwise gate kernel, even with the gate arithmetic removed. The copy specializes its loops on function constants where the shared kernel takes them as runtime arguments; we did not confirm that this is the cause, and dropped the fusion.

Convolution folded into the recurrence (rejected by estimate, not measured). In the proposed design each of the 1,536 recurrence threadgroups would recompute the causal convolution for the 256 q/k channels of its head; with 16 distinct q/k heads shared by 48 value heads, each unique channel would be recomputed 96 times, and the convolution state re-read as often. We judged the redundant work larger than the one dispatch it would save and did not implement it.

9 Discussion

What the levers are worth here. Of the 11.6 ms between llama.cpp’s forward and DS4’s, 4.2 ms are the switches of this paper, measured within one engine and one weight file; the rest is a comparison of complete engines, which the profiles of §4 attribute to dispatch count and CPU encoding but which we did not ablate across engines. The controlled experiment puts the format lever at 0.8 ms once the kernels are fused and at 2.8–6.9 ms when they are not, so the return from changing the format depends on the kernels it meets. GPU timestamps, CPU critical-path effects and context-dependent attention partitioning are not unique to unified memory; what is specific

to this setting is that at 27B scale on one GPU, per-dispatch cost and CPU gaps are each a few percent of the step and add up, and that a slow sampler is invisible to every GPU-side tool.

Hand-written schedules against frameworks. DS4 implements each family as a hand-written Metal schedule; the cost is that each new family is a port, the benefit is that fusions which depend on the model or the session (batching a layer’s side projections, scaling split-KV work to the context) are local changes with measured effects. llama.cpp and MLX both fuse some operators; we have not audited which, and the MLX result shows that a cheaper format on a general framework reaches a similar step time.

What is left. The optimized forward is 6.3 ms above the 24.3 ms streaming reference. Because the reference assigns the best observed rate to every matrix shape, that difference includes the slower matvecs as well as the latency-bound recurrence, the attention path, per-dispatch ramp and host work, and the measurements do not partition it exactly; §7.4 indicates that dequantization is a small term once the large matvecs are fused. The model ships a multi-token prediction block, so several candidate positions could be verified per weight pass; the benefit depends on acceptance rates, drafting cost and verifying the recurrent state, and we have not measured it.

10 Related work

Local inference runtimes. llama.cpp and ggml [8, 7] are the reference for consumer-hardware inference; DS4’s Q4_K and Q4_0 matvecs and its split-KV decode attention are ports and adaptations of theirs, and the fused kernels of §5 are new. MLX [10] is Apple’s array framework; mlx-lm [17] implements the Gated DeltaNet families.

Roofline and phase analyses. FlashAttention [3] made attention IO-aware for training and prefill; Flash-Decoding [4] partitions decode attention along the key sequence, which is the design the split-KV kernel here follows; disaggregated serving [18, 25] separates the prefill and decode phases at the cluster level. These target discrete GPUs and batched serving; we measure single-stream decode where the CPU shares the memory system.

Hybrid linear-attention models. Gated DeltaNet [23] and the Qwen3-Next and Qwen3.5 families [19, 20] interleave linear-attention layers with a minority of softmax-attention layers; the reference CUDA kernels [24] cannot be used on Metal, and the decode recurrence here is a direct implementation of the delta rule.

Quantization formats. Weight-only quantization methods [6, 15], the k-quant block formats [11], importance-matrix calibration [12] and mixed-recipe files [22] trade bytes for dequantization work; §7.4 measures that trade on an Apple GPU.

Speculative and multi-token decoding. Verifying several tokens per weight pass [13, 2, 1, 14] is the standard route past the streaming bound; the model used here ships a multi-token prediction block [9, 5].

11 Limitations

All results are from one machine, an M2 Ultra with a 60-core GPU; achieved rates and the balance between levers will differ on other generations (the M5 family adds GPU matrix units that change the prefill regime), and a device sweep is the obvious extension. The decomposition is for one dense hybrid family; the MoE result is a step time and a profile, not an ablation. The per-stage profiler perturbs what it measures by isolating stages, so we use it for rankings and deltas. Effective bandwidths are bytes over kernel time, not counter measurements; the mechanisms proposed for the kernel results are consistent with the data but were not isolated with microbenchmarks. The correctness checks are spot checks at eight points; perplexity is on forty chunks of one corpus; the sampler finding depends on the compiler and flags.

12 Conclusion

A decode step on Apple silicon is not simply bandwidth bound at the 27B scale: matvec kernel shape, dispatch count and CPU work between tokens each cost milliseconds, and each needs its own measurement. GPU timestamps per stage and a CPU inter-token split turned each optimization into a targeted change with a measured effect, and the switches make the account auditable: 12% of the step within one engine and weight file, and a sampler fix of 1.8 ms per token that no GPU-side tool could have found. The controlled format experiment shows where one lever stops and another starts: the cheaper block format is worth 8–16% against simple kernels and 2–3% against fused ones, so a quantization format has to be judged together with the kernels that read it.

References

- [1] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *International Conference on Machine Learning (ICML)*, 2024.
- [2] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. arXiv:2302.01318, 2023.
- [3] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [4] Tri Dao, Daniel Haziza, Francisco Massa, and Grigory Sizov. Flash-decoding for long-context inference. <https://crfm.stanford.edu/2023/10/12/flashdecoding.html>, 2023.
- [5] DeepSeek-AI. DeepSeek-V3 technical report. arXiv:2412.19437, 2024.
- [6] Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations (ICLR)*, 2023.
- [7] Georgi Gerganov and contributors. ggml: Tensor library for machine learning. <https://github.com/ggml-org/ggml>, 2023.

- [8] Georgi Gerganov and contributors. llama.cpp: LLM inference in C/C++. <https://github.com/ggml-org/llama.cpp>, 2023.
- [9] Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Rozière, David Lopez-Paz, and Gabriel Synnaeve. Better & faster large language models via multi-token prediction. In *International Conference on Machine Learning (ICML)*, 2024.
- [10] Awni Hannun, Jagrit Digani, Angelos Katharopoulos, and Ronan Collobert. MLX: Efficient and flexible machine learning on Apple silicon. <https://github.com/ml-explore/mlx>, 2023.
- [11] Kawrakow. k-quants: 2–6 bit quantization with per-block scales (llama.cpp pull request 1684). <https://github.com/ggml-org/llama.cpp/pull/1684>, 2023.
- [12] Kawrakow. Importance matrix calculation for quantization (llama.cpp pull request 4861). <https://github.com/ggml-org/llama.cpp/pull/4861>, 2024.
- [13] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning (ICML)*, 2023.
- [14] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE: Speculative sampling requires rethinking feature uncertainty. In *International Conference on Machine Learning (ICML)*, 2024.
- [15] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024.
- [16] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. arXiv:1609.07843, 2016.
- [17] MLX contributors. mlx-lm: LLM inference and fine-tuning with MLX. <https://github.com/ml-explore/mlx-lm>, 2024.
- [18] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative LLM inference using phase splitting. In *ACM/IEEE International Symposium on Computer Architecture (ISCA)*, 2024.
- [19] Qwen Team. Qwen3-next: Towards ultimate training and inference efficiency. <https://qwen.ai/blog?id=qwen3-next>, 2025.
- [20] Qwen Team. Qwen3.8-27b model card and configuration (revision 1d4bf0f2). <https://huggingface.co/Qwen/Qwen3.8-27B>, 2026.
- [21] Salvatore Sanfilippo and contributors. ds4 (DwarfStar): a model-specific LLM inference engine in C and Metal. <https://github.com/antirez/ds4>, 2026.
- [22] Unsloth. Unsloth dynamic 2.0 GGUFs. <https://docs.unsloth.ai/basics/unsloth-dynamic-2.0-ggufs>, 2025.
- [23] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving Mamba2 with delta rule. In *International Conference on Learning Representations (ICLR)*, 2025.

- [24] Songlin Yang and Yu Zhang. FLA: A Triton-based library for hardware-efficient implementations of linear attention mechanism. <https://github.com/fla-org/flash-linear-attention>, 2024.
- [25] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.

A Additional results

Configuration	ms/forward	Δ vs all on
all optimizations on	30.80 ± 0.04	
no fused gate/up SwiGLU	32.86 ± 0.01	+2.06
no multi-weight projections	32.14 ± 0.10	+1.34
no vectorized single-weight matvec	31.65 ± 0.03	+0.85
no K-split (FFN down)	31.70 ± 0.07	+0.90
fixed 32 split-KV workgroups	30.95 ± 0.02	+0.15
no fused residual+norm	30.92 ± 0.12	+0.12
multi-weight kernel, 2 rows/simdgroup	30.69 ± 0.03	-0.11
ggml-style Q4_K matvecs, generic fusions	34.64 ± 0.01	+3.84
all off	34.97 ± 0.01	+4.17

Table 8: Leave-one-out ablation, pure Q4_K, greedy, 20-token prompt, 64 tokens, generation loop per forward, mean of three runs.

Prompt tokens	ms per decode forward			prefill t/s		
	DS4 loop	MLX loop	llama.cpp fwd	DS4	MLX	llama.cpp
1450	31.4 ± 0.1	30.4 ± 0.0	42.5 ± 0.0	269 ± 0	221 ± 0	266 ± 0
4306	31.9 ± 0.1	31.6 ± 0.0	43.2 ± 0.0	273 ± 0	226 ± 0	266 ± 0
8590	32.6 ± 0.1	32.9 ± 0.0	43.9 ± 0.0	268 ± 0	224 ± 0	261 ± 0
17158	34.5 ± 0.0	34.1 ± 0.0	45.7 ± 0.1	258 ± 0	219 ± 0	250 ± 0
32866	38.1 ± 0.1	37.1 ± 0.0	49.4 ± 0.0	240 ± 0	208 ± 0	232 ± 0

Table 9: Context-length sweep, greedy, 64 decoded tokens, mean $\pm$ sd of three runs. DS4 and llama.cpp on pure Q4_K, MLX on its 4-bit affine conversion; all three engines read the same prompt files (hashes in `paper/logs/prompt_sha256.txt`) and their tokenizers agree on the counts shown. On the 32,866-token prompt mlx-lm reported 54 generated tokens before stopping at end of sequence; its mean is normalized by 53 decode forwards, so it covers a shorter decode window than the other engines.

A.1 Downstream check

As a coarse check on reasoning tasks we ran DS4’s built-in evaluation harness on its first ten questions (drawn from GPQA Diamond, SuperGPQA and AIME 2025) with thinking enabled, greedy decoding

and a 2,500-token reply budget. The harness forces the thinking block closed near the budget and extracts an answer from what follows, so a cut-off answer can still be correct. Pure Q4_K scored 8/10 (3 cut off), pure Q4.0 7/10 (3), Q8_0 7/10 (4): within one question of each other, which is the resolution of ten items, and not evidence of a ranking.

B Reproduction

The code is our fork of the DS4 repository [21]:

<https://github.com/parasol-aser/ds4>

Three branches build on each other. The sampler and the diagnostics are on `general-decode-fixes`; the model and its decode paths are on `qwen38-support`; this document and the scripts in `paper/scripts/` are on `paper-artifact`. The runs reported here are packaged under `paper/logs/`: the raw logs, the result set `results.json`, the model-file hashes in `sha256.txt`, and the hashes of the prompts and of the wikitext-2 archive and test split in `prompt_sha256.txt`. The downstream questions are built into `ds4-eval` at the pinned commit. The scripts write to `paper/scripts/out/`, and `tables.py` generates the table bodies from the result set. Everything below was run on macOS 26.1 with Xcode’s command-line tools; a llama.cpp checkout is needed for the baselines, the reference logits and the quantization. Every command is run from the repository root unless stated.

1. Build

```
git clone https://github.com/parasol-aser/ds4 && cd ds4 && git checkout paper-artifact
make                                     # ds4, ds4-server, ds4-agent, ds4-eval
git clone https://github.com/ggml-org/llama.cpp ~/github/llama.cpp
(cd ~/github/llama.cpp && git checkout 6d9c82ea2bb34e277c0664b8dd3434bfb4dcfb27 \
    && cmake -B build && cmake --build build -j)
pip3 install mlx==0.32.2 mlx-lm==0.31.3 huggingface_hub matplotlib numpy
```

2. Models

```
./download_model.sh qwen38-q8           # Qwen3.8-27B-Q8_0.gguf      (unsloth/Qwen3.8-27B-GGUF @ 4
    ca7207)
./download_model.sh qwen38-q4km         # Qwen3.8-27B-UD-Q4_K_M.gguf (same release)
./download_model.sh ds4f-q2             # DeepSeek V4 Flash mixed-recipe file (-0731 build)
paper/scripts/run_q40.sh                # fetches imatrix_unsloth.gguf, builds the pure Q4_K and
    Q4_0 files, prints SHA-256
```

SHA-256 of the files we measured (`paper/logs/sha256.txt`; `run_q40.sh` verifies them with `VERIFY_HASHES=1`):

File	SHA-256 (first 16 and last 8 hex digits)
Qwen3.8-27B-Q4_K.gguf	7d4154def38abac7...af9a9526
Qwen3.8-27B-Q4_0.gguf	129e216a45980381...0fc7a34e
Qwen3.8-27B-UD-Q4_K_M.gguf	322e194ff79741c7...39123482
Qwen3.8-27B-Q8_0.gguf	a680f44a06920e5d...9b67e348
imatrix_unsloth.gguf	0ee5b10bd0c2fa21...6599f1c1

The DeepSeek file we profiled is an earlier build of the `ds4f-q2` recipe (same name without the `-0731` suffix) that is no longer distributed; its step time in §7 should be re-measured on the current file.

3. Experiments

```
cd paper/scripts && mkdir -p out
curl -sL -o out/wikitext-2-raw-v1.zip \
  https://huggingface.co/datasets/ggml-org/ci/resolve/main/wikitext-2-raw-v1.zip
unzip -qo out/wikitext-2-raw-v1.zip -d out # zip sha256 ef7edb56...435a11; wiki.test.raw 173c87a5
...7dd08
python3 make_prompts.py # out/prompt_{1k,4k,8k,16k,32k}.txt from prompt_1439.txt
./run_all.sh # ~2 h: headline (5 runs), context (3 runs), ablation (3), profiles,
  PPL, MoE
./run_correctness.sh # ~15 min: reference logits at 9 (prompt, step) points, compiles
  logits_dump
./run_eval.sh # ~40 min: downstream check
python3 analyze.py # out/results.json
python3 tables.py # ../tables/*.tex (table bodies used by main.tex)
python3 figures.py # ../figs/{context,ablation,breakdown}.pdf
cd .. && pdflatex main && bibtex main && pdflatex main && pdflatex main
```

The scripts anchor paths on their own location, stop on the first missing prerequisite or failed command, and write every raw log to out/logs/. In our runs three measurements hit disk-bound stalls after the page cache had been turned over by other model files (one llama.cpp 16k-context run at 1.6 s per token, one DS4 Q8_0 prefill at 44 t/s, one Q4_0 run at 0.5 t/s); they were rerun with a warm cache and the packaged logs hold the reruns. The GPU must be otherwise idle: DS4 refuses to start while another DS4 process holds /tmp/ds4.lock, and timing is not meaningful with a second model process on the GPU. The short prompt is

```
<|im_start|>user
Write a short paragraph about the ocean.<|im_end|>
<|im_start|>assistant
<think>

</think>
```

(20 tokens); the 1,439-token prompt is paper/scripts/prompt_1439.txt.

4. Switches and profilers

```
DS4_METAL_DECODE_STAGE_PROFILE=1 DS4_METAL_CB_TIMES=1 # per-stage GPU spans (stage_agg.py
  aggregates)
DS4_TOKEN_TIMING=1 # CLI: sample / emit / forward per token; per-forward ms
  when greedy
DS4_GLM_LOGIT_DUMP=f DS4_QWEN_LOGIT_DUMP_STEP=k # logits of step k (0 = prefill output)
DS4_QWEN_DISABLE_FUSED_SWIGLU=1 # fused gate/up -> two matvecs + SwiGLU
DS4_QWEN_DISABLE_MULTI_PROJ=1 # multi-weight proj -> one launch each
DS4_QWEN_DISABLE_FAST_MV=1 # vectorized single-weight mv -> ggml-style
DS4_METAL_Q4K_KSPLIT_NSQ=0 # K-split (FFN down) off
DS4_METAL_QWEN_FA_NWG=32 # fixed 32 split-KV workgroups
DS4_QWEN_DISABLE_FUSED_NORM=1 # residual+norm fusion off
DS4_METAL_Q4K_MULTI_NRO=2 # multi-weight kernel rows per simdgroup
```

The Metal System Trace captures of §4 were recorded and exported with:

```
xcrun xctrace record --template 'Metal System Trace' --time-limit 120s \
  --output ds4.trace --launch -- ./ds4 -m gguf/Qwen3.8-27B-UD-Q4_K_M.gguf -p "$PROMPT" -n 32 --temp
  0
xcrun xctrace export --input ds4.trace \
  --xpath '/trace-toc/run[@number="1"]/data/table[@schema="metal-gpu-intervals"]'
xcrun xctrace export --input ds4.trace \
```

```
--xpath '/trace-toc/run[@number="1"]/data/table[@schema="metal-application-command-buffer-  
submissions"]'
```

GPU busy fraction and per-token GPU time come from the union of the GPU intervals in the decode window; CPU encode time per token from the submission table's encoder durations.